PHASE 2 · AGENT GIT (CLOUD)

Phase 2·Git — Route planning and trip cost

You can show after it: Trip step Route: three routes with full cost, breaks and no-entry waits; choose one; ETAs fill in; driver gets a WhatsApp summary.

RUN WITH	BRANCH	TIME · MODEL
Agent Git — claude.ai/code, cloud (Account 2)	`git/phase2-routes`	≈ 3 h · Sonnet

Starts after	Runs at the same time as	Builds (Architecture section 8)
Merge 1 (tag v1)	2·Code — Loader link, route service, live map	F16 Route planning and trip cost

1. Your lane and the other agent's lane

	You — 2·Git	Other agent — 2·Code
Builds	F16 Route planning and trip cost	F13 Loader link and loader page (no login) F14 Route service (Edge Function route-plan) F15 Live map
Writes only in	`src/features/routes`.	`src/features/{loader,live-map}`, `supabase/functions/route-plan`, `supabase/sql/phase2code_*.sql`.

Never touch: `src/features/{loader,live-map}`, `supabase/functions/route-plan` (Agent Code). Call route-plan only through its contract; mock it in tests. Need something from the other lane? Use the fallback in your build plan or write a CONTRACT CHANGE REQUEST (Engineering Reference 12.4).

2. Already built when you start

- Project, database (+ real POIs), contracts, module + slot system, router.
- Sign-in pages, accounts, demo trucks for every new company.
- Design system incl. Hindi button, `td()` demo-data translation, NextStepCard, StepTabs, BigTile, MapView.
- Shell (rail / bottom tabs, top bar with Hindi button, bell, avatar menu), Home with slots `home.actions` + `home.widgets`, Settings with slot `settings.sections`.
- Trucks (/trucks, slot `truck.tabs`), Trips (/trips, trip page with StepTabs, slots `trip.steps` + `trip.actions`), Cargo list (slot `cargo.addMethods`).
- Packing engine + 3D load plan (public API pack, fitAdditional, TruckScene, TopView, SideView, usePlanForTrip); dark map style in `public/map/`.

Read before planning: CLAUDE.md, docs/PROGRESS.md, the Plan PDF (sections 0, 1, 4 and your features in 8), Design System sections 2–6, 7.10, 8, 13, Engineering Reference sections 1, 2, 3, 6, 7, 8, and the code in `src/app`, `src/components/common`, `src/types/domain.ts` and the `index.ts` of features you use.

3. Start the session

1. Open **claude.ai/code** in the browser (Account 2 — keeps its limit separate from Agent Code).

2. Repository `spareway`, branch `main`, environment `spareway-cloud`, **Plan mode**, model as shown, paste the prompt.

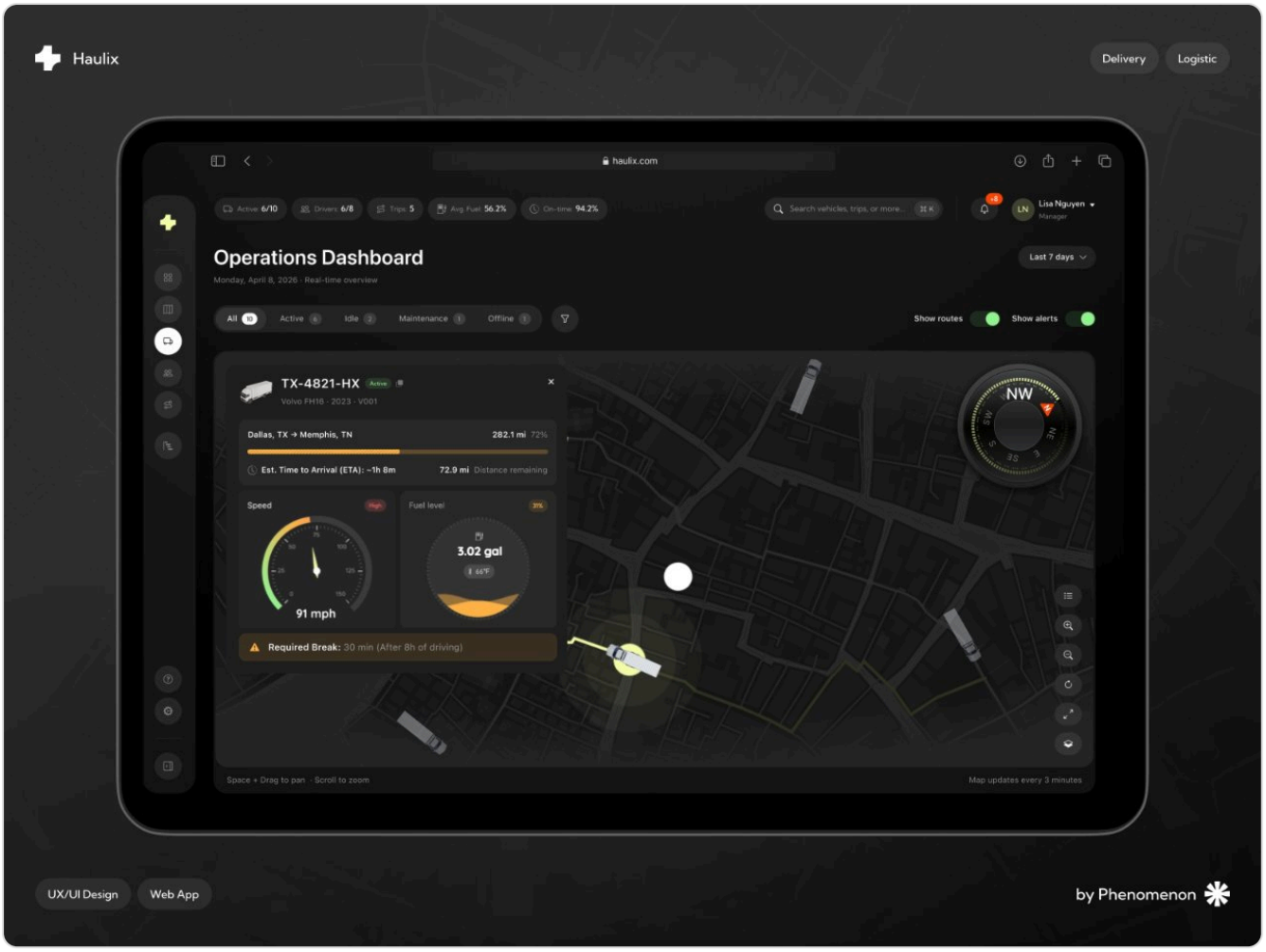
```
You are Agent Git (cloud session) for Spareway. Start Phase 2·Git. Read
docs/Spareway_01_Plan_Architecture_and_Spec.pdf, docs/Spareway_02_Design_System.pdf and
docs/Spareway_03_Engineering_Reference.pdf (all pages, in parts) and
docs/phases/Phase2_Git_Route_Planning_and_Trip_Cost.pdf. Also read CLAUDE.md and
docs/PROGRESS.md. Create branch git/phase2-routes from main and push only there. You have
no Supabase, secrets or real phone: never try to connect; use the committed types,
fixtures and mocks. Ask me your questions first, then show your plan. Do not write code
before I approve the plan.
```

4. Questions Claude must ask first

1. Default departure when a trip has none (default tomorrow 6:00 am)?
2. Rest rules: Settings values (default) or Vansh's numbers?
3. Driver message in Hindi by default (default) or English?

5. Build plan (in this order)

1. **Logic + tests first**: chainage, tolls, checkpoints, planSchedule, costRoute on a fixture line through the demo stops + POIs.
2. **Route step** as Design System 7.10 with fixture mode (mocked route-plan responses from Engineering Reference 6.1).
3. **Journey list + driver message** + WhatsApp share; slot `trip.steps`.
4. “Code must verify” list; Hindi; push.



Haulix reference 6 — Live map with selected-truck card, gauges, break banner — map look for the Route step.

6. Check before saying done

- ☐ `npm run check` and `npm run build` pass with zero errors.
- ☐ Logic tests pass; the cheapest route is marked Best; no-entry wait computed for Mumbai.
- ☐ Route step renders in fixture mode; Choose writes the right fields (mocked api).
- ☐ Design checklist; “Code must verify” list written.
- ☐ Every new screen tested with the Hindi button (labels + demo data).
- ☐ Design System section 13 check done.

7. Hand-off

1. Update the Phase 2·Git section of `docs/PROGRESS.md` (Engineering Reference 12.2): built, how to show it, open points.
2. Write the **“Code must verify”** list: what you couldn't test without the database, keys or a phone — exact clicks and expected results.
3. Add assumptions to `docs/DECISIONS.md`.
4. Push `git/phase2-routes` (do not merge to main).
5. Tell the user “Phase 2·Git ready to merge” + the 3 most important things to check.